

Viva: Project 18, Joke Machine

API: JokeAPI (<https://jokeapi.dev>) - Key: No key needed

Brief: Random jokes by category with a "next joke" button, using safe mode.

30 questions, roughly easy to hard. Sections A and B are common ground for every project; C and D are specific to this one. Ask 8 to 12 in a typical 10 minute viva.

A. Kotlin and Android basics

1. What is the difference between val and var, and which did you use for your Retrofit instance?
2. What is a data class, and what does Kotlin generate for you?
3. A joke can be one line or a setup plus delivery. How did you model both shapes in Kotlin?
4. What does ? mean on a property, and which joke fields are genuinely optional?
5. Which Android component holds the current joke, and what happens on rotation?
6. Would a sealed class be a better fit than nullable fields here? Explain your reasoning.
7. Where do you declare the INTERNET permission, and what happens if you forget it?

B. Retrofit, coroutines and networking

8. What problem does Retrofit solve for you?
9. Explain the role of the converter factory in your Retrofit builder.
10. The category sits in the path while flags sit in the query string. Which annotations did you use for each?
11. Why is your API call function marked suspend?
12. Which dispatcher runs the network call, and which updates the UI?
13. What happens if you call the API on the main thread?
14. What is a coroutine scope, and which one did you launch from?
15. What happens if the user taps "next joke" ten times rapidly?

C. The JokeAPI and this app

16. What is the exact endpoint, and what does the URL look like for a programming joke?
17. What does safe-mode do, and why is it required for a classroom project?
18. What is the blacklistFlags parameter, and how does it differ from safe mode?
19. If you omit safe mode, what kind of content could reach a student's screen? Whose responsibility is that?
20. The response has a type field of single or twopart. How does your UI branch on it?
21. For a two part joke, how did you reveal the punchline, immediately or on tap?
22. The response includes an error boolean. Do you check it before reading the joke?
23. What are the available categories, and how did you let the user pick one?
24. JokeAPI rate limits to 120 requests per minute per IP. Is that a real constraint for your app?

D. Edge cases, design and extensions

25. What does your app show while a joke is loading?
26. A user has no connection and taps next. Describe exactly what they see.

27. What stops the same joke appearing twice in a row?
28. How would you add a share button for the current joke?
29. How would you save favourite jokes so they survive app restarts?
30. How would you preload the next joke so the button feels instant?

Humble Coders project viva preparation. These are the kinds of questions you should be able to answer about the app you built. Knowing the list is not the same as knowing your own code.